

CPP PISCINE NOTES VIDÉOS

<https://elearning.intra.42.fr/notions/piscine-c-d00-c-basics/subnotions>
<https://www.learncpp.com/>

<https://en.cppreference.com/w/>
<https://cplusplus.com/reference/>

notes on intra video provided for the CPP piscine

DAY 00: BASICS	2
1. NAMESPACE	2
2. STDIO STREAMS	3
3. CLASS AND INSTANCES	4
4. MEMBER ATTRIBUTES AND MEMBER FUNCTIONS	5
5. THIS	6
6. LIST INITIALIZATION	7
7. CONSTANTS	9
8. VISIBILITY	10
9. CLASS VS. STRUCT	11
10. ACCESSORS	12
11. COMPARISONS	13
12. NON MEMBER ATTRIBUTES AND NON MEMBER FUNCTIONS	14
13. POINTER TO MEMBERS // A FINIR	15
DAY 01: NEW	16
14. NEW AND DELETE	16
15. RÉFÉRENCES	17
16. FILESTREAM	19
DAY 02: CANONICAL FORM	20
17. ADHOC POLYMORPHISM (SURCHARGE DE FONCTION)	20
18. OPERATOR OVERLOAD	21
19. CANONICAL FORM	23
DAY 03: INHERITANCE	25
20. INHERITANCE C++	25
DAY 04: POLYMORPHISM / ABSTRACT	28
21. SUB-TYPING POLYMORPHISM	28
22. ABSTRACT CLASSES AND INTERFACES	29
DAY 05: EXCEPTIONS	30
23. NESTED CLASSES	30
24. EXCEPTIONS	31
DAY 06: CASTS	38
25. C TYPE CONVERSION	39
26. C TYPE REINTERPRETATION	39
27. C QUALIFIER REINTERPRETATION	39
28. UPCAST AND DOWNCAST	39
29. STATIC CAST	40
30. DYNAMIC CAST	40
31. REINTERPRET CAST	41
32. CONST CAST	41
33. CAST OPERATORS	42
34. EXPLICIT KEYWORD	42
DAY 07: TEMPLATES	43
35. FROM C - PARAMETRIC MACROS	43
36. TEMPLATES	43
37. DEFAULT TYPE	44
38. SPECIALIZATION	45
DAY 08: STL	46
39. STL	46

DAY 00: BASICS

1. NAMESPACE

- namespace est un sous scope, permet d'avoir def diverses pour memes noms
- :: est un **opérateur de résolution de portée** => fait appel à une var au sein d'un namespace
- gl_var correspond a ::gl_var
- Foo::gl_var différent de Bar::gl_var
- **Aliasing** de namespace Muf = Bar => initialise Muf avec les mêmes propriétés que Bar

```
//main.cpp
#include <stdio.h>

int      gl_var = 1;
int      f(void){return 2;}

namespace Foo
{
    int      gl_var = 3;
    int      f(void){return 4;}
}

namespace Bar
{
    int      gl_var = 5;
    int      f(void){return 6;}
}

namespace Muf = Bar;

int main()
{
    printf("Bar::gl_var: %d\n", Bar::gl_var);
    printf("Bar::f(): %d\n", Bar::f());
    ...
}
```

output:

```
gl_var: 1
f(): 2
::gl_var: 1
::f(): 2
Foo::gl_var:3
Foo::f():4
Bar::gl_var:5
Bar::f():6
Muf::gl_var:5
Muf::f():6
```

2. STDIO STREAMS

- cin, cout sont des **objets**
- << >> double chevrons permettent de gérer ces flux de données
- std::endl = retour à la ligne
- std est un namespace

```
#include <iostream>

int main()
{
    char buf[512];

    std::cout << "Hello world!"<< std::endl;
    // injecte Hello world dans cout
    std::cout << "Input a word: ";
    std::cin >> buff;
    // recupere donnees depuis cin, et stocke dans buff
    std::cout << "You entered: [" << buff << "]" << std::endl;

    return 0;
}
```

3. CLASS AND INSTANCES

- Sample est une **classe** (partie statique), comme un modèle, un template
- instance est une **instance de classe** (partie dynamique), variable basée sur le modèle Sample
- **déclaration** classe.hpp avec ses fonctions et attributs
- **définition** classe.cpp

//Sample.class.hpp

- attention au point virgule a la fin

```
#ifndef SAMPLE_CLASS_H
# define SAMPLE_CLASS_H
```

```
class Sample
{
    public:
        Sample(void);
        ~Sample(void);
};
```

```
#endif
```

//Sample.class.cpp

- pas de type de retour pour les constructeur et destructeur, ce sont des procedures
- Classe::Variable/Fonction() !difference avec namespace ?

```
#include <iostream>
#include "Sample.class.hpp"
```

```
Sample::Sample(void) // constructor
{
    std::cout << "Constructor called" << std::endl;
    return;
}
```

```
Sample::~~Sample(void) // destructor
{
    std::cout << "Destructor called" << std::endl;
    return;
}
```

// main.cpp

```
#include "Sample.class.hpp"
```

```
int main()
{
    Sample instance;
    // ceci est une instantiation de classe, ca revient a une déclaration de var, instance est une instance de la
    classe Sample
    // va (1) créer une var de type Sample appelée instance
    // (2) exécuter le code du constructeur
    return 0;
}

// ici la fin du scope va appeler le code du destructeur car c'est une var locale, donc détruite en fin de scope
```

compilation: g++ -Wall -Wextra -Werror -Wno-unused Sample.class.cpp main.cpp

==> output

```
Constructor called
Destructor called
```

4. MEMBER ATTRIBUTES AND MEMBER FUNCTIONS

- **attribut membre** = variable qui appartient à la classe, utilisable à partir d'une instance
- **fonction membre** = fonction qui appartient à la classe, utilisable à partir d'une instance
- fonction membre a toujours un paramètre invisible : un pointer sur l'instance courante

//Sample.class.hpp

```
#ifndef SAMPLE_CLASS_H
# define SAMPLE_CLASS_H

class Sample
{
    public:
        int    foo; // déclaration d'un attribut membre

        Sample(void);
        ~Sample(void);

        void    bar(void); // déclaration d'une fonction membre
};

#endif
```

//Sample.class.cpp

```
#include <iostream>
#include "Sample.class.hpp"

void    Sample::bar(void) // définition de la fonction bar
{
    std::cout << "Member function bar called"<< std::endl;
    return ;
}

Sample::Sample(void) // constructor
{
    std::cout << "Constructor called" << std::endl;
    return;
}

Sample::~~Sample(void) // destructor
{
    std::cout << "Destructor called" << std::endl;
    return;
}
```

// main.cpp

```
#include "Sample.class.hpp"

int main()
{
    Sample instance; // instanciation de la classe Sample

    instance.foo = 42; // instance est une variable de pile donc on utilise le .
    std::cout << "instance.foo " << instance.foo << std::endl;

    instance.bar(); // appelle la fonction bar de l'instance de la classe Sample

    return 0;
}
```

compilation: g++ -Wall -Wextra -Werror -Wno-unused Sample.class.cpp main.cpp

==> output

```
Constructor called
]instance.foo 42
Member function bar called
Destructor called
```

5. THIS

- this est un **pointeur special d'instance** qui pointe vers **l'instance courante**
- this fait référence à l'instance courante de la classe au sein du code de la classe
- rappel de syntaxe pour ajouter des attributs (variables dans la classe) et des fonctions membre (fonctions dans la classe)

```
//Sample.class.hpp
#ifndef SAMPLE_CLASS_H
#define SAMPLE_CLASS_H

class Sample
{
public:
    int    foo; // déclaration d'un attribut membre

    Sample(void);
    ~Sample(void);

    void    bar(void); // déclaration d'une fonction membre
};

#endif

//Sample.class.cpp
#include <iostream>
#include "Sample.class.hpp"

void    Sample::bar(void) // définition de la fonction bar
{
    std::cout << "Member function bar called"<< std::endl;
    return ;
}

Sample::Sample(void) // constructor
{
    std::cout << "Constructor called" << std::endl;

    this->foo = 42; // pointeur vers l'instance courante
                  // -> car this est un pointeur, on déréfère le pointeur pour accéder au membre foo
    std::cout << "this->foo: " << this->foo << std::endl;

    this->bar(); // on appelle la fonction directement à partir du pointeur

    return;
}

Sample::~Sample(void) // destructor
{
    std::cout << "Destructor called" << std::endl;
    return;
}

// main.cpp
#include "Sample.class.hpp"

int main()
{
    Sample instance; // instantiation de la classe Sample

    return 0;
}
```

compilation: g++ -Wall -Wextra -Werror -Wno-unused Sample.class.cpp main.cpp

==> output

```
Constructor called
this.foo 42
Member function bar called
Destructor called
```

6. LIST INITIALIZATION

```
//Sample1.class.hpp
#ifndef SAMPLE1_CLASS_H
# define SAMPLE1_CLASS_H

class Sample1
{
public:
    char    a1;
    int     a2;
    float   a3;

    Sample1(char p1, int p2, float p3);
    ~Sample1(void);
};

#endif

//Sample1.class.cpp
#include <iostream>
#include "Sample1.class.hpp"
// constructor
Sample1::Sample1(char p1, int p2, float p3) // éviter cette méthode
{
    std::cout << "Constructor called" << std::endl;

    this->a1 = p1;
    std::cout << "this->a1: " << this->a1 << std::endl;

    this->a2 = p2;
    std::cout << "this->a2: " << this->a2 << std::endl;

    this->a3 = p3;
    std::cout << "this->a3: " << this->a3 << std::endl;

    return;
}
// destructor
Sample1::~Sample1(void)
{
    std::cout << "Destructor called" << std::endl;
    return;
}

//Sample2.class.hpp
#ifndef SAMPLE2_CLASS_H
# define SAMPLE2_CLASS_H

class Sample2
{
public:
    char    a1;
    int     a2;
    float   a3;

    Sample2(char p1, int p2, float p3);
    ~Sample2(void);
};

#endif

//Sample2.class.cpp
#include <iostream>
#include "Sample2.class.hpp"

// constructor
Sample2::Sample2(char p1, int p2, float p3) : a1(p1), a2(p2), a3(p3) // liste d'initialisation, à privilégier
{
    std::cout << "Constructor called" << std::endl;
    std::cout << "this->a1: " << this->a1 << std::endl;
    std::cout << "this->a2: " << this->a2 << std::endl;
    std::cout << "this->a3: " << this->a3 << std::endl;

    return;
}
// destructor
Sample2::~Sample2(void)
{
    std::cout << "Destructor called" << std::endl;
    return;
}

// main.cpp
#include "Sample.class.hpp"
```

```
int main()
{
    Sample1 instance1('a', 42, 4.2f);
    Sample2 instance2('z', 13, 3.14f);

    return 0;
}
```

compilation: g++ -Wall -Wextra -Werror -Wno-unused Sample1.class.cpp Sample2.class.cpp main.cpp

==> output

```
Constructor called
this.a1 a
this.a2 42
this.a3 4.2
Constructor called
this.a1 z
this.a2 13
this.a3 3.14
Destructor called
Destructor called
```


7. CONSTANTS

- **Const** rend une **variable constante** (différence entre un prog qui explose et un prog qui n'explose pas)
- avec CPP privilégier l'utilisation de constante au max car langage complexe
- par définition on ne peut pas faire **d'assignation** à une constante, la seule manière d'attribuer une valeur à une constante c'est **d'initialiser** la constante avec cette valeur

```
//Sample.class.hpp
#ifndef SAMPLE_CLASS_H
# define SAMPLE_CLASS_H

class Sample
{
public:
    float const pi; // attribut const
    int      qd;

    Sample(float const f); // constructeur prend un float constant
    ~Sample(void);

    void      bar(void) const;
};

#endif

//Sample.class.cpp
#include <iostream>
#include "Sample.class.hpp"

// définition de la fonction bar en ajoutant mot clef const
void      Sample::bar(void) const
{
    std::cout << "Member function bar called"<< std::endl;
    std::cout << "this->pi : " << this->pi << std::endl;
    std::cout << "this->qd : " << this->qd << std::endl;
    return ;
}

// constructor
Sample::Sample(float const f) : pi(f), qd(42)
{
    std::cout << "Constructor called" << std::endl;
    return;
}

// destructor
Sample::~Sample(void)
{
    std::cout << "Destructor called" << std::endl;
    return;
}

// main.cpp
#include "Sample.class.hpp"

int main()
{
    Sample instance; // instantiation de la classe Sample

    return 0;
}

compilation: g++ -Wall -Wextra -Werror -Wno-unused Sample.class.cpp main.cpp
==> output
```

8. VISIBILITY

- **public/private** :contrôler **l'encapsulation des membres de la classe**, les attributs et fonctions membres visibles uniquement depuis l'intérieur de la classe ou depuis l'extérieur
- **public**: accessible de l'intérieur ou de l'extérieur
- **private**: accessible uniquement depuis l'intérieur de la classe
utiliser underscore "*_NomMembre*" par convention pour savoir si c'est un membre private
garder en private tous les membres qui ne concernent pas l'utilisateur

```
//Sample.class.hpp
#ifndef SAMPLE_CLASS_H
#define SAMPLE_CLASS_H

class Sample
{
public: // accessible de l'intérieur ou de l'extérieur
    int    publicFoo;

    Sample(void);
    ~Sample(void);

    void    PublicBar(void) const;

private: // accessible uniquement depuis l'intérieur de la classe
    int    _privateFoo;

    void    _privateBar(void) const;
};

#endif

//Sample.class.cpp
#include <iostream>
#include "Sample.class.hpp"

// Member function
void    Sample::bar(void) const
{
    std::cout << "Member function bar called"<< std::endl;
    return ;
}

// constructor
Sample::Sample(void)
{
    std::cout << "Constructor called" << std::endl;

    this->publicFoo = 0;
    std::cout << "this->publicFoo: " << this->publicFoo << std::endl;

    this->_privateFoo = 0;
    std::cout << "this->_privateFoo: " << this->_privateFoo << std::endl;

    this->PublicBar();
    this->_privateBar();

    return;
}

// destructor
Sample::~Sample(void)
{
    std::cout << "Destructor called" << std::endl;
    return;
}

// main.cpp
#include <iostream>
#include "Sample.class.hpp"

int main()
{
    Sample instance; // instantiation de la classe Sample

    instance.publicFoo = 42;
    std::cout << "instance.publicFoo: " << instance.publicFoo << std::endl;

    instance.PublicBar();
    return 0;
    // on ne peut pas utiliser les membres private ici
}
```

9. CLASS VS. STRUCT

- même syntaxe qu'une classe, structures et classes fonctionnent de la même manière
 - la seule différence en c++, c'est que une structure a un scope par défaut public, et la classe a un scope par défaut private
- ⇒ utiliser de préférence des classes pour sécuriser le code

```
//Sample1.class.hpp // déclaration d'une structure, par défaut public
```

```
#ifndef SAMPLE1_CLASS_H  
# define SAMPLE1_CLASS_H
```

```
struct Sample1  
{  
    int    Foo;  
  
    Sample1(void);  
    ~Sample1(void);  
  
    void    Bar(void) const;  
};
```

```
#endif
```

```
//Sample2.class.hpp // déclaration d'une classe, par défaut private
```

```
#ifndef SAMPLE2_CLASS_H  
# define SAMPLE2_CLASS_H
```

```
struct Sample2  
{  
    int    Foo;  
  
    Sample1(void);  
    ~Sample1(void);  
  
    void    Bar(void) const;  
};
```

```
#endif
```

```
// main.cpp
```

```
#include <iostream>  
#include "Sample1.class.hpp"  
#include "Sample2.class.hpp"
```

```
int main()  
{  
    Sample1 instance1; // instanciation de la structure  
    Sample2 instance2; // instanciation de la classe  
  
    return 0;  
}
```

10. ACCESSORS

- permet d'avoir un contrôle sur ce que l'on fournit à l'utilisateur
- permet d'avoir un contrôle sur ce que l'utilisateur peut modifier
- a vraiment privilégier pour sécuriser le code

//Sample.class.hpp

```
#ifndef SAMPLE_CLASS_H
# define SAMPLE_CLASS_H
```

```
class Sample
```

```
{
```

```
public:
```

```
    Sample(void);
```

```
    ~Sample(void);
```

```
    int     getFoo(void) const; // fonctions publique pour accéder à la variable private, const car pas de modif
```

```
    void     setFoo(int v);
```

```
private:
```

```
    int     _Foo;
```

```
};
```

```
#endif
```

//Sample.class.cpp

```
#include <iostream>
```

```
#include "Sample.class.hpp"
```

```
// constructor
```

```
Sample::Sample(void)
```

```
{
```

```
    std::cout << "Constructor called" << std::endl;
```

```
    this->setFoo(0);
```

```
    std::cout << "this->getFoo(): " << this->getFoo() << std::endl;
```

```
    return;
```

```
}
```

```
// destructor
```

```
Sample::~~Sample(void)
```

```
{
```

```
    std::cout << "Destructor called" << std::endl;
```

```
    return;
```

```
}
```

```
// Member function
```

```
int     Sample::getFoo(void) const // accesseur
```

```
{
```

```
    return this->_foo;
```

```
}
```

```
void     Sample::setFoo(int v) // accesseur, permet de mettre des regles de modifications avec securites
```

```
{
```

```
    if (v >= 0)
```

```
        this->_foo = v;
```

```
    return;
```

```
}
```

// main.cpp

```
#include <iostream>
```

```
#include "Sample.class.hpp"
```

```
int main()
```

```
{
```

```
    Sample instance; // instantiation de la classe
```

```
    instance.setFoo(42);
```

```
    std::cout << "instance.getFoo(): " << instance.getFoo() << std::endl; // est updaté
```

```
    instance.setFoo(-42);
```

```
    std::cout << "instance.getFoo(): " << instance.getFoo() << std::endl; // est ignoré
```

```
    return 0;
```

```
}
```

11. COMPARISONS

- comparaison entre les structures

```
//Sample.class.hpp
#ifndef SAMPLE_CLASS_H
# define SAMPLE_CLASS_H
```

```
class Sample
{
public:
    Sample(int v); // constructeur
    ~Sample(void); // destructeur

    int    getFoo(void) const; // accesseur
    int    compare(Sample * other) const; // fonction membre qui va faire une comparaison

private:
    int    _Foo;
};
#endif
```

```
//Sample.class.cpp
```

```
#include <iostream>
#include "Sample.class.hpp"
// constructeur
Sample::Sample(int v) : _foo(v)
{
    std::cout << "Constructor called" << std::endl;
    return;
}
// destructeur
Sample::~~Sample(void)
{
    std::cout << "Destructor called" << std::endl;
    return;
}
// accesseur
int    Sample::getFoo(void) const
{
    return this->_foo;
}
// Member function for comparison
int    Sample::compare(Sample * other) const
{
    if (this->_foo < other->getFoo()) // on utilise l'accession à partir de l'instance extérieure passée en
parametre
        return -1;
    else if (this->_foo > other->getFoo())
        return 1;
    return 0;
}
// main.cpp
#include <iostream>
#include "Sample.class.hpp"

int main()
{
    Sample instance1(42);
    Sample instance2(42);

    if (instance1.compare(&instance2) == 0)
        std::cout << "instance 1 and instance 2 are structurally equal"<< std::endl;
    else
        std::cout << "instance 1 and instance 2 are not structurally equal"<< std::endl;

    return 0;
}
```

12. NON MEMBER ATTRIBUTES AND NON MEMBER FUNCTIONS

- variables et fonctions **d'instance (membres)**
 - existe au niveau de l'instance de la classe, il faut avoir instancier la classe
 - chaque attribut va être différent dans chaque instance
- variables et fonctions **de classe (non membres)**
 - existent au niveau de la classe et non pas de l'instance
 - paramètre de l'instance courante ne passe pas : on ne peut pas faire appel au pointer "*this*" dans ces fonctions
- mot clef **static** a une nouvelle sémantique

```
//Sample.class.hpp
#ifndef SAMPLE_CLASS_H
#define SAMPLE_CLASS_H

class Sample
{
public:
    Sample(int v); // constructeur
    ~Sample(void); // destructeur

    static int    getNbInst(void); // déclaration fonction non membre

private:
    static int    _nbInst;          // déclaration attribut non membre
                                   // compteur du nombre d'instance : ce type de données n'a de sens qu'au niveau de
    la classe et non de l'instance // private donc on peut pas accéder de l'extérieur
};
#endif

//Sample.class.cpp
#include <iostream>
#include "Sample.class.hpp"
// constructeur
Sample::Sample(void)
{
    std::cout << "Constructor called" << std::endl;
    Sample::_nbInst += 1;

    return;
}
// destructor
Sample::~Sample(void)
{
    std::cout << "Destructor called" << std::endl;
    Sample::_nbInst -= 1;

    return;
}
// fonction accesseur non membre, paramètre de l'instance courante n'est pas passé
int    Sample::getNbInst(void)
{
    return Sample::_nbInst; // on utilise directement le nom de la classe et non this
}
//
int    Sample::_nbInst = 0; // n'est pas initialisé dans un constructeur, donc on doit ajouter une initialisation
distincte, c'est la seule façon d'initialiser un attribut non membre, statique
```

13. POINTER TO MEMBERS // A FINIR

- deux nouveaux types
 - pointer sur membre
 - pointer sur fonction membre

//Sample.class.hpp

```
#ifndef SAMPLE_CLASS_H
# define SAMPLE_CLASS_H
```

```
class Sample
{
public:
    int    foo;

    Sample(void); // constructeur
    ~Sample(void); // destructeur

    void    bar(void) const;

};
#endif
```

//Sample.class.cpp

```
#include <iostream>
#include "Sample.class.hpp"
```

// constructeur

```
Sample::Sample(void) : foo(0)
```

```
{
    std::cout << "Constructor called" << std::endl;
    return;
}
```

// destructeur

```
Sample::~Sample(void)
```

```
{
    std::cout << "Destructor called" << std::endl;
    return;
}
```

// fonction membre

```
int    Sample::bar(void) const
```

```
{
    std::cout << "Member function bar called" << std::endl;
    return;
}
```

// main.cpp

```
#include <iostream>
#include "Sample.class.hpp"
```

```
int main()
```

```
{
    Sample instance;
    Sample *instancep = &instance; // declaration d'un poiteur sur la pile en donnant l'adresse d'une autre
variable

    int    Sample::*p = NULL; // pointer sur un entier qui est membre de la classe
    void    (Sample::*f)(void) const;

    p = &Sample::foo; // on assigne au pointer l'adresse du membre foo de la classe sample

    std::cout << "Value of member foo: " << instance.foo << std::endl;
    instance.*p = 21; // on assigne une nouvelle valeur au contenu du pointer p, de l'instance instance
                    // nouvel opérateur ".*" pour les instances de pile
    std::cout << "Value of member foo: " << instance.foo << std::endl;
    instance->*p = 42; // opérateur "->*" pour les pointeurs
    std::cout << "Value of member foo: " << instance.foo << std::endl;

    f = &Sample::bar;

    (instance.*f)();
    (instancep->*f)();

    return 0;
}
```

DAY 01: NEW

14. NEW AND DELETE

- malloc naze pour allocation d'objet, car n'appelle pas constructeur ou destructeur.
- new/ delete appellent constructeur et destructeur

```
#include <string>

class Student
{
    private:
        std::string _login;
    public:
        Student(std::string login) : _login(login)
        {
            std::cout << "Student "<< this->_login << " is born" << std::endl;
        }
        Student() : _login("ldefault")
        {
            std::cout << "Student "<< this->_login << " is born" << std::endl;
        }
        ~Student()
        {
            std::cout << "Student "<< this->_login << " died" << std::endl;
        }
};

int main()
{
    Student    bob = Student("bfubar");
    Student*   jim = new Student("jfubar");
    Student*   students = new Student [42];

    // do some stuff here

    delete    jim; // kim is destroyed
    delete[]  students; // kim is destroyed

    return (0); // bob is destroyed
}
```


15. RÉFÉRENCES

- référence = alias vers une variable existante
- référence = pointeur **constant**, **toujours déréférencé**, et **jamais nul**
- **complète les pointeurs mais ne les remplacent pas**
- **obligation d'initialiser la référence donc parfois besoin d'utiliser les pointeurs**

```
#include <iostream>

int main()
{
    int        numberOfBalls    = 42;
    int*        ballsPtr        = &numberOfBalls;
    int&        ballsRef        = numberOfBalls;
    int&        ballsRef2;      // IMPOSSIBLE
    std::cout << numberOfBalls << " " << *ballsPtr << " " << ballsRef << std::endl;
    *ballsPtr = 21;              // change la valeur de numberOfBalls
    std::cout << numberOfBalls << std::endl;
    ballsRef = 84;              // change la valeur de numberOfBalls
    std::cout << numberOfBalls << std::endl;

    return (0);
}
```

- Passage de paramètre par référence

```
#include <iostream>
#include <string>

void byPtr(std::string* str)
{
    *str += "and ponies";
}

void byConstPtr(std::string const *str)
{
    std::cout << *str << std::endl;
}

void byRef(std::string& str)
{
    str += "and ponies";
}

void byConstRef(std::string const & str)
{
    std::cout << str << std::endl;
}

int main()
{
    std::string str = "I like butterflies";
    std::cout << str << std::endl;
    byPtr(&str);
    byConstPtr(&str);

    str = "I like otters";

    std::cout << str << std::endl;
    byRef(str);
    byConstRef(str);

    return (0);
}
```

- Retour par référence

```
#include <iostream>
```

```

#include <string>

class Student
{
private:
    std::string    _login;
public:
    Student(std::string const & login) : _login(login)
    {
    }
    std::string & getLoginRef()
    {
        return this->_login;
    }
    std::string const & getLoginRefConst() const
    {
        return this->_login;
    }
    std::string*    getLoginPtr()
    {
        return &(this->_login);
    }
    std::string const * getLoginPtrConst() const
    {
        return &(this->login);
    }
};

int    main()
{
    Student    bob = Student("bfubar");
    Student const    jim = Student("jfubar");
    std::cout << bob.getLoginRefConst() << " " << jim.getLoginRefConst() << std::endl;
    std::cout << *(bob.getLoginPtrConst()) << " " << *(jim.getLoginPtrConst()) << std::endl;
    bob.getLoginRef() = "bobfubar";
    std::cout << bob.getLoginRefConst() << std::endl;
    *(bob.getLoginPtr()) = "bobbyfubar";
    std::cout << bob.getLoginRefConst() << std::endl;
}

```

16. FILESTREAM

- voir cpp.com pour plus d'info sur maniere de gerer stream
- pas besoin d'utiliser open et close comme en c
- stringstream, istream, ostream (comme printf)

```
#include <iostream>
#include <fstream>
```

```
int main()
{
    std::ifstream ifs("numbers");
    unsigned int dst;
    unsigned int dst2;

    ifs >> dst >> dst2;
    std::cout << dst << " " << dst2 << std::endl;
    ifs.close();
    //...
    std::ofstream ofs("test.out");
    ofs << "I like ponies a whole damn lot" << std::endl;
}
```

DAY 02: CANONICAL FORM

17. ADHOC POLYMORPHISM (SURCHARGE DE FONCTION)

- definition d'une même fonction avec plusieurs types d'arguments
- fonction pour n'importe quelle fonction

```
#ifndef SAMPLE_CLASS_H
# define SAMPLE_CLASS_H

class Sample
{
    public:
        Sample();
        ~Sample();
        void bar(char const c) const;
        void bar(int const n) const;
        void bar(float const z) const;
        void bar(Sample const &i) const;
};
```

18. OPERATOR OVERLOAD

- étendre syntaxe et sémantique de cpp en définissant de nouveaux opérateurs
- surcharger opérateurs
- tres casse gueule, règles à respecter
 - surcharge d'opérateur soit naturelle, éviter comportement bizarre
 - surcharge d'opérateur ait un rapport avec sémantique naturelle
 - ne pas en faire, seuls cas où c'est justifié sont rares surcharge de () (fonctor)

+ 1 1 (notation préfixe) - notation fonctionnelle + (1, 1)

1.+ (1) (notation préfixe) - instance et fonction membre à qui on envoie le paramètre

1 + 1 (notation infixe)

1 1 + (notation postfixe)

```
// INTEGER.HPP
#ifndef INTEGER_CLASS_H
#define INTEGER_CLASS_H

#include <iostream>

class Integer
{
public:
    Integer(int const n);
    ~Integer(void);

    int  getValue(void) const;

    Integer & operator=(Integer const & rhs); // surcharge d'opérateur
    Integer operator+(Integer const & rhs) const;
private:
    int  _n;
};

std::ostream & operator<<(std::ostream & o, Integer const & rhs);

#endif
// INTEGER.CPP
#include <iostream>
#include "Integer.class.hpp"

Integer::Integer(int const n) : _n(n)
{
    std::cout << "Constructor called with value" << std::endl;
    return;
}

Integer::~~Integer(void)
{
    std::cout << "Destructor called" << std::endl;
    return;
}

int  Integer::getValue(void) const;
{
    return this->_n;
}

Integer & Integer::operator=(Integer const & rhs)
{
    std::cout << "Assigation operator called from " << this->_n;
    std::cout << " to " << rhs.getValue() << std::endl;
    this->_n = rhs.getValue();
    return *this;
}
```

```

Integer    Integer::operator+(Integer const & rhs) const;

{
    std::cout << "Addition operator called from " << this->_n;
    std::cout << " and " << rhs.getValue() << std::endl;
    return Integer(this->_n + rhs.getValue());
}

std::ostream &    operator<<(std::ostream &o, Integer const &rhs)
{
    o << rhs.getValue();
    return o;
}

// MAIN
#include <iostream>
#include "Integer.class.hpp"
int    main()
{
    Integer    x    (30);
    Integer    y    (10);
    Integer    z    (0);

    std::cout << "Value of x : " << x << std::endl;
    std::cout << "Value of y : " << y << std::endl;
    y = Integer(12);
    std::cout << "Value of y : " << y << std::endl;
    std::cout << "Value of z : " << z << std::endl;
    z = x + y;
    std::cout << "Value of z : " << z << std::endl;
}

```

19. CANONICAL FORM

- best practice:
 - constructeur par défaut
 - constructeur par copie
 - opérateur d'assignation
 - destructeur
- déclarer des classes de la même façon, utilisation unifiée de toutes les classes

```
// SAMPLE.HPP
#ifndef SAMPLE_CLASS_H
#define SAMPLE_CLASS_H

class Sample
{
public:
    Sample      ( void );           // constructeur par défaut
    Sample      ( int const n );    // constructeur par parametre
    Sample      ( Sample const & src); // constructeur par copie
    ~Sample     ( void );           // destructeur
    Sample & operator= (Sample const & rhs); // operateur assignation
    int         getFoo   ( void ) const;
    std::string // afficher la classe, serialiser
private:
    int         _foo;
};

std::ostream & operator<< (std::ostream &o, Sample const & i);
#endif
```

```
// SAMPLE.CPP
#include <iostream>
#include "Sample.class.hpp"

Sample::Sample(void) : _foo(0)
{
    std::cout << "Default constructor called" << std::endl;
    return;
}

Sample::Sample(int const n) : _foo(n)
{
    std::cout << "Parametric constructor called" << std::endl;
    return;
}

Sample::Sample(Sample const & src)
{
    std::cout << "Copy constructor called" << std::endl;
    *this = src;
    return;
}

Sample::~Sample(void)
{
    std::cout << "Destructor called" << std::endl;
    return;
}

Sample::Sample(void) : _foo(0);
{
    std::cout << "Default constructor called" << std::endl;
    return;
}
```

```
// MAIN.CPP
#include <iostream>
#include "Sample.class.hpp"
```

```
int main()
```

```
{  
    Sample    instance1;  
    Sample    instance2(42);  
    Sample    instance3(instance1);  
  
    std::cout << instance1 << std::endl;  
    std::cout << instance2 << std::endl;  
    std::cout << instance3 << std::endl;  
  
    instance3 = instance2;  
  
    std::cout << instance3 << std::endl;  
    return 0;  
}
```


DAY 03: INHERITANCE

20. INHERITANCE C++

- **base class / parent class / super class** vs. **derived class/child class/ sub class**

The derived class inherits all the properties of the base class, without changing the properties of base class and may add new features to its own. The derived class is the specialized class for the base class.

Note: A derived class doesn't inherit access to private data members. However, it does inherit a full parent object, which contains any private members which that class declares.

```
syntax:
class <derived_class_name> : <inheritance access-specifier> <base_class_name>
{
    Base class member access specifier
    public:
    protected: // accessible by inherited but not by outside
    private:

//body
}
```

Order of instantiation

Derived is really two parts: a Base part, and a Derived part. When C++ constructs derived objects, it does so in phases. First, the most-base class (at the top of the inheritance tree) is constructed first. It then walks through the inheritance tree in order and constructs each successive derived class. Each child class is constructed in order, until the most-child class (at the bottom of the inheritance tree) is constructed last.

```
Output:
Instantiating Base
Base constructor

Instantiating Derived
Base constructor
Derived constructor

//
Constructing A: A
Constructing B: A B
Constructing C: A B C
Constructing D: A B C D
```

Modes of Inheritance

access-specifier — either of private, public or protected. If neither is specified, PRIVATE is taken as default

Base class member access specifier	Type of inheritance		
	Public	Protected	Private
Public	Public	Protected	Private (public members of the base class can only be accessed by the member functions of the derived class. They are inaccessible to the objects of the derived class)
Protected	Protected		
Private	Not accessible (hidden)		

Types Of Inheritance:

```
- Single inheritance
class subclass_name : access_mode base_class
{
    // body of subclass
};
```

- Multiple inheritance

```
class subclass_name : access_mode base_class1, access_mode base_class2, ....
{
    // body of subclass
};
```

- Multilevel inheritance (a derived class is created from another derived class.)

```
class C
{
    ... ..
};
class B:public C
{
    ... ..
};
class A: public B
{
    ... ..
};
```

- Hierarchical inheritance

```
class A
class B : public A
class C : public A
class D : public B
class E : public B
class F : public C
class G : public C
```

- Hybrid inheritance (combines more than one type of inheritance i.e. hierarchical and multiple)

```
class A
class G
class B : public A
class D : public B
class E : public B
class C : public A, public class G
```

Multipath hybrid inheritance is a special case where there is a need to use scope resolution operator or virtual base class to clarify the ambiguity

héritage en diamant is also a special case that may indicate a problem in the code

⇒ utiliser virtual sur les meres pour éviter la double creation de la grandmere

```
class Quadruped // can access name, run (), and legs
{
    private:
        std::string name; // only accessible from a quadruped object
    protected:
        Leg legs[4];
    public:
        void run(int distance);
}
```

```
class Dog : public Quadruped // can access run () and legs
{};
```

```
int main() // can only access run ()
{}
```

```
# include <string>
```

```
class Animal
{
    public:
        Animal();
        Animal(Animal const &);
```

```

    Animal& operator=(Animal const &);
    ~Animal();

    void run(int distance);
    void walk(int distance);
    void cal();
    void eat(std::string const & what);

```

```

private:
    int _numberOfLegs;

```

```

// new
class Cat : public Animal
{
public:
    Cat ();
    Cat(Cat const &);
    Cat& operator=(Cat const &);
    ~Cat();

    void run(int distance); // overrides base class function
    void scornSomeone(std::string const & target); // comportement propre
}

```

```

class Pony : public Animal
{
public:
    Pony();
    Pony(Pony const &);
    Pony& operator=(Pony const &);
    ~Pony();

```

```

    void doMagic(std::string const & target);
}

```

```

// old
class Cat
{
public:
    Cat ();
    Cat(Cat const &);
    Cat& operator=(Cat const &);
    ~Cat();

```

```

    void run(int distance);
    void scornSomeone(std::string const & target);
private:
    int _numberOfLegs;
};

```

```

class Pony
{
public:
    Pony();
    Pony(Pony const &);
    Pony& operator=(Pony const &);
    ~Pony();

```

```

    void run(int distance);
    void doMagic(std::string const & target);
private:
    int _numberOfLegs;
}

```

DAY 04: POLYMORPHISM / ABSTRACT

21. SUB-TYPING POLYMORPHISM

- **méthode** (fonction membre virtuel / à lien dynamique) : vérifie le type réel de l'objet
- virtual: résolution de l'appel de fonction devient dynamique = faite à l'exécution
- make base pointers call the derived version of a function instead of the base version

```
# include <string>
# include <iostream>

class Character
{
public:
    void sayHello(std::string const & target);
    //virtual void sayHello(std::string const & target); // solution tells the
program to go look and see if there are any more-derived versions of the function available
between Base and Derived
};

class Warrior : public Character
{
public:
    void sayHello(std::string const & target);
    //virtual void sayHello(std::string const & target); // solution
};

class Cat
{
    //[]
};

void Character::sayHello(std::string const & target)
{
    std::cout << "Hello " << target << " !" << std::endl;
}

void Warrior::sayHello(std::string const & target)
{
    std::cout << "Fuck off "<< target << ", I dont't like you !" << std::endl;
}

int main()
{
    Warrior* a = new Warrior(); // OK
    Character* b = new Warrior(); // OK, warrior is a character
    // type reel: warrior
    Warrior* c = new Character(); // NOK, character is not a warrior
    Character* c = new Cat(); // NOK, cat is not a character
    a->sayHello("students");
    b->sayHello("students");
}
```

Output:

```
// normal
>> Fuck off students // a->sayHello => Warrior
>> Hello students ! // b->sayHello => Character
// virtual
>> Fuck off students // a->sayHello => Warrior
>> Fuck off students // b->sayHello => Warrior
```

22. ABSTRACT CLASSES AND INTERFACES

Abstract classes

- **pure virtual function = abstract function**
- pure virtual function simply acts as a placeholder that is meant to be redefined by derived classes
- To create a pure virtual function, rather than define a body for the function, we simply assign the function the value 0
- it is up to the derived classes to implement this function
- any class with one or more pure virtual functions becomes an abstract base class, which means that it can not be instantiated and forces the derived class to instantiate these functions
- any derived class must define a body for this function, or that derived class will be considered an abstract base class as well.
- A pure virtual function is useful when we have a function that we want to put in the base class, but only the derived classes know what it should return.
- Just like with normal virtual functions, pure virtual functions can be called using a reference (or pointer) to a base class:
- It turns out that we can create pure virtual functions that have definitions:
- This paradigm can be useful when you want your base class to provide a default implementation for a function, but still force any derived classes to provide their own implementation. However, if the derived class is happy with the default implementation provided by the base class, it can simply call the base class implementation directly.

Interface classes

- An interface class is a class that has **no member variables**, and where **all of the functions are pure virtual!** In other words, the class is purely a definition, and has no actual implementation. Interfaces are useful when you want to define the functionality that derived classes must implement, but leave the details of how the derived class implements that functionality entirely up to the derived class.
- Any class inheriting from Interface must provide implementations for all three functions in order to be instantiated. Now, let's say you need to write some code that uses an error log. If you write your code so it includes FileErrorLog or ScreenErrorLog directly, then you're effectively stuck using that kind of error log (at least without recoding your program). A much better way to implement this function is to use the interface class instead:
- Don't forget to include a virtual destructor for your interface classes, so that the proper derived destructor will be called if a pointer to the interface is deleted.
-

```
class ACharacter // prefixe pour signifier que classe abstraite
{
    virtual void    attack(std::string const & target) = 0;
```

```
>> Fuck off students    // a->sayHello => Warrior
```

DAY 05: EXCEPTIONS

23. NESTED CLASSES

```
class Cat
{
    public:
        class Leg
        {
            // [...]
        };
};

class Dog
{
    public:
        class Leg
        {
            // [...]
        };
};

int main()
{
    Cat      somecat;
    Cat::Leg somecatsleg;
}
```

24. EXCEPTIONS

- nouvelle maniere de gerer les erreurs, monter un message à travers la pile d'appel
- new, open, plein de fonctions renvoient des exceptions

```
throw -1; // throw a literal integer value
throw ENUM_INVALID_INDEX; // throw an enum value
throw "Can not take square root of negative number"; // throw a literal C-style (const char*)
string
throw dx; // throw a double variable that was previously defined
throw MyException("Fatal Error"); // Throw an object of class MyException
```

```
try
{
    // Statements that may throw exceptions you want to handle go here
    throw -1; // here's a trivial throw statement
    std::cout << "This never prints\n";
}
```

```
catch (int x)
{
    // Handle an exception of type int here
    std::cerr << "We caught an int exception with value" << x << '\n';
}
```

- Try blocks and catch blocks work together -- a try block detects any exceptions that are thrown by statements within the try block, and routes them to the appropriate catch block for handling. A **try block must have at least one catch block immediately following it, but may have multiple catch blocks listed in sequence.**
- Once an exception has been caught by the try block and routed to a catch block for handling, the exception is considered handled, and **execution will resume as normal after the catch block.**
- Catch parameters work just like function parameters, with the parameter being available within the subsequent catch block. Exceptions of fundamental types can be caught by value, but exceptions of non-fundamental types should be caught by const reference to avoid making an unnecessary copy.
- Just like with functions, if the parameter is not going to be used in the catch block, the variable name can be omitted.

What catch blocks typically do

If an exception is routed to a catch block, it is considered “handled” even if the catch block is empty. However, typically you’ll want your catch blocks to do something useful. There are three common things that catch blocks do when they catch an exception:

- First, catch blocks may print an error (either to the console, or a log file).
- Second, catch blocks may return a value or error code back to the caller.
- Third, **a catch block may throw another exception.** Because the catch block is outside of the try block, the newly thrown exception in this case is not handled by the preceding try block -- it’s handled by the next enclosing try block.

Catch-all handlers

- no avoid program to stop when we use functions that we don't know how they are implemented, so we don't know if they throw a specific type of exception that we do not handle
- use of Ellipsis
- Often, the catch-all handler block is left empty, this will catch any unanticipated exceptions and prevent them from stack unwinding to the top of your program, but does no specific error handling.
- One interesting use for the catch-all handler is to wrap the contents of main():

```
int main()
{
    try
    {
        runGame();
    }
    catch(...)
    // or
    // catch (const std::exception& exception)
    {
        std::cerr << "Abnormal termination\n";
    }
    saveState(); // Save user's game
    return 1;
}
```

Exceptions and member functions

Up to this point in the tutorial, you've only seen exceptions used in non-member functions. However, exceptions are equally useful in member functions, and even more so in overloaded operators.

Handlers for derived exception classes should be listed before those for base classes.

Using the standard exceptions directly

- Nothing throws a `std::exception` directly, and neither should you. However, you should feel free to throw the other standard exception classes in the standard library if they adequately represent your needs. You can find a list of all the standard exceptions on [cppreference](#).
- `std::runtime_error` (included as part of the `stdexcept` header) is a popular choice, because it has a generic name, and its constructor takes a customizable message:

```
throw std::runtime_error("Bad things happened");
```

Deriving your own classes from `std::exception` or `std::runtime_error`

You can, of course, derive your own classes from `std::exception`, and override the virtual `what()` const member function. Here's the same program as above, with `ArrayException` derived from `std::exception`:

```
class ArrayException : public std::exception
{
private:
    std::string m_error{}; // handle our own string
public:
    ArrayException(std::string_view error): m_error{error}{}
    // std::exception::what() returns a const char*, so we must as well

    const char* what() const noexcept override
    // Note that virtual function what() has specifier noexcept (which means the function promises not to
    // throw exceptions itself). Therefore, our override should also have specifier noexcept.
    {
        return m_error.c_str();
    }
};

class IntArray
{
private:
    int m_data[3] {};
    // assume array is length 3 for simplicity
public:
    IntArray() {}
    int getLength() const { return 3; }
    int& operator[](const int index)
    {
        if (index < 0 || index >= getLength())
            throw ArrayException("Invalid index");
        return m_data[index];
    }
};

int main()
{
    IntArray array;
    try
    {
        int value{ array[5] };
    }
    catch (const ArrayException& exception) // derived catch blocks go first
    {
        std::cerr << "An array exception occurred (" << exception.what() << ")\n";
    }
    catch (const std::exception& exception)
    {
        std::cerr << "Some other std::exception occurred (" << exception.what() <<
        ")\n"; }
}
```


Because `std::runtime_error` already has string handling capabilities, it's also a popular base class for derived exception classes. Here's the same example derived from `std::runtime_error` instead:

```
#include <exception> // for std::exception
#include <iostream>
#include <stdexcept> // for std::runtime_error
#include <string>
class ArrayException : public std::runtime_error
{
public:
    // std::runtime_error takes a const char* null-terminated string.
    // std::string_view may not be null-terminated, so it's not a good choice here.
    // Our ArrayException will take a const std::string& instead,
    // which is guaranteed to be null-terminated, and can be converted to a const char*.
    ArrayException(const std::string& error) : std::runtime_error{ error.c_str() }
    // std::runtime_error will handle the string { }
    // no need to override what() since we can just use std::runtime_error::what()
};

class IntArray
{
private:
    int m_data[3]{};
    // assume array is length 3 for simplicity
public:
    IntArray() {}
    int getLength() const { return 3; }
    int& operator[](const int index)
    {
        if (index < 0 || index >= getLength())
            throw ArrayException("Invalid index");
        return m_data[index];
    }
};

int main()
{
    IntArray array;
    try
    {
        int value{ array[5] };
    }
    catch (const ArrayException& exception) // derived catch blocks go first
    {
        std::cerr << "An array exception occurred (" << exception.what() << ")\n";
    }
    catch (const std::exception& exception)
    {
        std::cerr << "Some other std::exception occurred (" << exception.what() << ")\n";
    }
}
```

It's up to you whether you want

- to create your own standalone exception classes,
- to use the standard exception classes, or
- to derive your own exception classes from `std::exception` or `std::runtime_error`.

All are valid approaches depending on your aims.

Rethrowing exceptions

Occasionally you may run into a case where you want to catch an exception, but not want to (or have the ability to) fully handle it at the point where you catch it. This is common when you want to **log an error**, but pass the issue along to the caller to actually handle.

different from one caught:

```
int getIntValueFromDatabase(Database* d, std::string table, std::string key)
{
    assert(d);
    try
    {
        return d->getIntValue(table, key);
        // throws int exception on failure
    }
}
```

```

    }
    catch (int exception)
    {
        // Write an error to some global logfile
        g_log.logError("getIntValueFromDatabase failed");
        throw 'q';
    }
    // throw char exception 'q' up the stack to be handled by caller of getIntValueFromDatabase()
}

```

exact same exception as the one that was just caught:

use the throw keyword from within the catch block (with no associated variable)

This throw keyword that doesn't appear to throw anything in particular actually re-throws the exact same exception that was just caught. No copies are made, meaning we don't have to worry about performance killing copies or slicing.

If rethrowing an exception is required, this method should be preferred over the alternatives.

```

#include <iostream>

class Base
{
public:
    Base() {}
    virtual void print()
    {
        std::cout << "Base";
    }
};

class Derived: public Base
{
public:
    Derived() {}
    void print() override
    {
        std::cout << "Derived";
    }
};

int main()
{
    try
    {
        try
        {
            throw Derived{};
        }
        catch (Base& b)
        {
            std::cout << "Caught Base b, which is actually a ";
            b.print();
            std::cout << '\n';
            throw;
            // note: We're now rethrowing the object here
        }
    }
    catch (Base& b)
    {
        std::cout << "Caught Base b, which is actually a ";
        b.print(); std::cout << '\n';
    }
    return 0;
}

```

function try blocks

When construction of an object fails, the destructor of the class is not called. Consequently, you may be tempted to use a function try block as a way to clean up a class that had partially allocated resources before failing. However, referring to members of the failed object is considered undefined behavior since the object is “dead” before the catch block executes. This means that **you can't use function try to clean up after a class**. If you want to clean up after a class, follow the standard rules for cleaning up classes that throw exceptions (see the “When constructors fail” subsection of lesson [20.5 -- Exceptions, classes, and inheritance](#)).

Function try is useful primarily for either **logging failures before passing the exception up the stack**, or for **changing the type of exception thrown**.

```
#include <iostream>

class A
{
private:
    int m_x;
public:
    A(int x) : m_x{x}
    {
        if (x <= 0)
            throw 1; // Exception thrown here
    }
};

class B : public A
{
public:
    B(int x) try : A{x} // note addition of try keyword here
    {
    }
    catch (...) // note this is at same level of indentation as the function itself
    {
        // Exceptions from member initializer list or constructor body are caught here
        std::cerr << "Exception caught\n";
        throw;
        // rethrow the existing exception
    }
};

int main()
{
    try
    {
        B b{0};
    }
    catch (int)
    {
        std::cout << "Oops\n";
    }
}
```

Inside a normal catch block, we have three options:

- We can throw a new exception,
- rethrow the current exception,
- or resolve the exception (by letting control reach the end of the catch block).

A function-level catch block for a non-value (void) returning function has the same set of options: throw, rethrow, or resolve.

However, a **function-level catch block for a constructor, destructor, or value-returning function** must either throw a new exception or rethrow the existing exception. Catch blocks for these types of functions are not allowed to resolve exceptions!

If control reaches the end of a function-level catch block, the behavior depends on the type of function.

Because such behavior varies for depending on the type of function, we recommend always explicitly throwing or rethrowing, and only letting control reach the end of the catch block for non-value returning functions when you're resolving the exception.

To cleanup previously allocated memory after a throw error is executed, a solution would be to use a local variable of a class that knows how to cleanup itself when it goes out of scope (often called a "smart pointer"). The standard library provides a class called `std::unique_ptr` that can be used for this purpose. `std::unique_ptr` is a template class that holds a pointer, and deallocates it when it goes out of scope.

```

#include <stdexcept>

void test1()
{
    try // essaye du code
    {
        // do some stuff
        if (/* there's an error */)
        {
            throw std::exception(); // si condition est rencontrée, va remonter dans
les blocs jusqu'à trouver un bloc catch
        }
        else
        {
            // do some stuff
        }
    }
    catch (std::exception e)
    {
    }
}

-----
void test2()
{
    try
    {
        // do some stuff
        if (/* there's an error */)
        {
            throw std::exception();
        }
        else
        {
            // do some stuff
        }
    }
}

-----
void test3()
{
    try
    {
        test2();
    }
    catch (std::exception& e) // catcher par reference
    {
        // Handle error
    }
}

-----
void test4()
{
    class PEBKACException : public std::exception
    {
    public:
        virtual const char * what () const throw()
        {
            return ("Problem exists");
        }
    };
    try
    {
        test3();
    }
}

```

```
}  
catch (PEBKACEException& e) // catch spécifique  
{  
    // Handle this specific error  
}  
catch (std::exception& e) // catch générique  
{  
    // Handle other errors that are like std::exception  
}  
}
```

DAY 06: CASTS

5 actions de cast

- type conversions (converting a value from one data type to another data type) `(int)var => static_cast<int>(var)`
 - Numeric promotions (type conversion of a narrower numeric type (such as a `char`) to a wider numeric type (typically `int` or `double`))
 - Numeric conversions (may result in the loss of data)
 - Arithmetic conversions
 - Other conversions
- type reinterpretation
- qualifier reinterpretation
- upcast de classe
- downcast de classe

Cast	Conv	reint	Upcast	Downcast	Type Qual.
Implicit	Yes		Yes		
static_cast	Yes		Yes	Yes	
dynamic_cast			Yes	Yes	
const_cast					Yes
reinterpret_cast		Yes	Yes	Yes	
Legacy C cast	Yes	Yes	Yes	Yes	Yes

Cast	Semantic checks	Reliable at run	Tested at run
Implicit	Yes	Yes	
static_cast	Yes		
dynamic_cast	Yes	Yes	Yes
const_cast			
reinterpret_cast			
Legacy C cast			

25. C TYPE CONVERSION

```
int    a = 42;
double b = a;
double c = (double) a;
```

```
int    d = a;
int    e = d;           // conversion implicite
int    f = (int) d;      // conversion explicite
```

- '-Wno-conversion' flag qui empêche la compilation en cas de **perte de précision** implicites (conversions implicites)
- hiérarchie entre type en fonction de precision

26. C TYPE REINTERPRETATION

- conversion identitaire = reinterpretation = les bits ne sont pas reorganisés, permet un jeu d'adresses
- void * est un type plus général
- promotion : plus précise -> plus générale
- demotion : plus générale -> plus précise
- on privilégie toujours une syntaxe explicite lors de demotion

```
float  a      = 420.042f;           // =    420.041992           // référence

void*  b      = &a;                  // =                               // implicit reinterpretation cast
void*  c      = (void *) &a;         // =                               // implicit reinterpretation cast

void *  d      = &a;                  // =                               // implicit promotion -> OK
int*    e      = d;                  // =    1137837408          // implicit demotion -> Hazardous
int*    f      = (int *) d;          // =    1137837408          // explicit demotion -> OK
```

27. C QUALIFIER REINTERPRETATION

- cas particulier de reinterpretation: bit restent les mêmes
- type qualifier : **const**, volatil
- adresse entier constant est plus générale que adresse d'entier non constant
- on ne peut pas aller d'une adresse constante a une adresse non constante
- si besoin de faire ça soit mauvais design, a refaire, soit code externe

```
int    a      = 42;                  // =    420.041992           // référence

int const * b  = &a;                  // =                               // implicit type qualifier cast
int const * c  = (int const *) &a;    // =                               // explicit type qualifier cast

int const * d  = &a;                  // =                               // implicit promotion -> OK
int*    e      = d;                  // =    1137837408          // implicit demotion -> doesn't compile
int*    f      = (int *) d;          // =    1137837408          // explicit demotion -> OK
```

28. UPCAST AND DOWNCAST

- nouvelle hiérarchie de type avec la notion d'héritage des classes

```
#include <iostream>
```

```
class Parent{};
class Child1: public Parent {};
class Child2: public Parent {};
```

```
int    main()
{
```

```

Child1      a;                                // reference value

Parent *    b    =&a;                        // implicit reinterpretation cast
Parent *    c    =(Parent *) &a;            // explicit reinterpretation cast

Parent *    d    =&a;                        // implicit upcast ->ok
Child1      e    =d;                        // implicit downcast -> no
Child2      f    =(Child2 *) d;            // explicit downcast -> ok

return 0;
}

```

29. STATIC CAST

- cast le plus simple et le plus usuel
- permet de faire des conversions simples entre valeurs directes
- upcast et downcast sur une hiérarchie de classe connues et empêche des casts sur des classes non liées
- protège: s'assure que le cast ne se fasse qu'entre adresses compatibles dans un même arbre d'héritage

```

int main (void)
{
    int      a      =42;                    // reference value
    double b      =a;                      // implicit promotion -> ok
    int      c      =b;                    // implicit demotion -> no
    int      d      = static_cast<int>(b);  // explicit demotion-> ok

    return 0;
}

```

```
#include <iostream>
```

```

class Parent{};
class Child1: public Parent {};
class Child2: public Parent {};

```

```
class Unrelated {};
```

```

int      main()
{
    Child1      a;                                // reference value

    Parent *    b    =&a;                        // implicit upcast
    Child1 *    c    =b;                        // implicit downcast -> no
    Child2 *    d    = static_cast<Child2 *>(b)  // Explicit downcast ->ok

    Unrelated*  e    =static_cast<Unrelated *>(&a); // Explicit conversion ->no

    return 0;
}

```

30. DYNAMIC CAST

- seul cast a avoir lieu au run time (a l'exécution), les autres a la compilation
- un programme peut compiler mais dynamic cast peut échouer à l'exécution => nouveau moyen de détecter les erreurs de ce type
- ne fonctionne que dans des cas de polymorphisme par sous-typage: au moins l'une des fonctions membre est virtuelle (une méthode)
- ne fonctionne que sur un pointeur ou une référence

- pertinent notamment lors de la gestion des plugins, pour vérifier la bonne compatibilité

```
#include <iostream>
#include <typeinfo>
#include <exception>
```

```
class Parent {public: virtual ~Parent(){};};
class Child1: public Parent {};
class Child2: public Parent {};
```

```
int main ()
{
```

```
    Child1      a;
    Parent *    b    = &a;
```

```
// explicit downcast POINTEUR
```

```
Child1 * c = dynamic_cast<Child1 *>(b); // retourne soit NULL soit adresse convertie
if (c == NULL)
```

```
{
    std::cout << "conversion is NOT ok" << std::endl;
}
else
{
    std::cout << "conversion is OK" << std::endl;
}
```

```
// explicit downcast REFERENCE
```

```
try
{
    Child2 & d = dynamic_cast<Child2 &>(*b);
    std::cout << "conversion is OK" << std::endl;
}
catch (std::bad_cast &bc)
{
    std::cout << "conversion is NOT ok : " << bc.what() << std::endl;
    return 0;
}
return 0;
}
```

31. REINTERPRET CAST

```
int main()
{
    float a = 420.042f;    // reference value

    void * b = &a;        // Implicit promotion -> ok
    int * c = reinterpret_cast<int *>(b);    // Explicit demotion -> ok
    int & d = reinterpret_cast<int &>(b);    // Explicit demotion -> ok
}
```

32. CONST CAST

- ne pas utiliser, mieux vaut transformer le code

```
int main()
{
    int a = 42;                // reference value

    int const * b = &a;        // Implicit promotion -> ok
    int * c = b;               // implicit demotion -> NO
```

```
int *      d = const_cast<int *>(b);      // Explicit demotion -> ok
}
```

33. CAST OPERATORS

```
class Foo
{
    public:
        Foo (float const v): _v(v){}
        float getV(){return this->_v;}
        operator float(){return this->_v;}
        operator int(){return static_cast<int>(this->_v);}
    private:
        float _v;
};

int main()
{
    Foo a(420.042f);
    float b = a;
    int c = a;

    std::cout << a.getV();
    std::cout << b;
    std::cout << c;
}
```

34. EXPLICIT KEYWORD

- interdit les conversions implicites

```
class A {};
class B {};

class C
{
    public:
        C(A const & _){return;}
        explicit C(B const & _){return;}
};

void f(C const & _){return;}

int main()
{
    f(A());
    f(B());
}
```

DAY 07: TEMPLATES

35. FROM C - PARAMETRIC MACROS

```
# define max(x, y) (((x) >= (y)) ? (x) : (y))
```

Tout ce qui commence par un # est une instruction pour le préprocesseur effets de bord

36. TEMPLATES

librairie standard cpp stl
.hpp

```
template <typename T >           // template definition
T max (T const & x, T const & y)
{
    return((x>y)? x: y);
}
```

```
int main()
{
    max<int>(a, b)           // explicit instantiation
    max<float>(a, b)         // explicit instantiation
    max(a, b)                // implicit instantiation
}
```

```
-----
template <typename T >
class List
{
    public:
        List<T>(T const & content){}
        List<T>(List <T> const & list){}
        ~List<T>(T const & content){}
    private:
        T      _content;
        List<T> _next;
};
```

```
int      main(void)
{
    List<int>      a(42);
    List<float>    b(3.14f);
    List<List<int>>> c(a);

    return 0;
}
```

```
-----
template <typename T , typename U>

List<int, float>  a(42, 3.14f);
```

37. DEFAULT TYPE

`template <typename T = float > // type par défaut, utilisé quand aucun autre type n'est spécifié`

`class Vertex`

```
{
    public:
        Vertex(T const & x, T const & y, T const & z) : _x(x), _y(y), _z(z) {}
        ~Vertex(){}

        T const & getX() const {return this->_x;}
        T const & getY() const {return this->_y;}
        T const & getZ() const {return this->_z;}

    private:
        T const _x;
        T const _y;
        T const _z;

        Vertex();
};
```

`template <typename T>`

`std::ostream & operator <<( std::ostream &o, Vertex<T> const & v)`

```
{
    std::cout.precision(1);
    o << settiosflags(std::ios::fixed);
    o << "Vertex( ";
    o << v.getX() << " ";
    o << v.getY() << " ";
    o << v.getZ() << " ";
    return o;
}
```

`int main ()`

```
{
    Vertex<int>    v1(12, 23, 34);
    Vertex        v2(12, 23, 34); // instantiation avec le type par défaut du template, ici conversion implicite

    std::cout << v1; // print int
    std::cout << v2; // print float

    return 0;
}
```

38. SPECIALIZATION

affine le comportement du code contenu dans le template
a du sens pour économiser de la memoire dans certains cas

```
template <typename T, typename U>
class Pair
{
    public:
        Pair<T, U>(T const & lhs, U const & rhs) : _lhs(lhs), _rhs(rhs)
        {
            std::cout << "Generic";
            return;
        }
        ~Pair<T, U>(){}

        T const & fst() const {return this->_lhs;}
        U const & snd() const {return this->_rhs;}

    private:
        T const & _lhs;
        U const & _rhs;

        Pair<T, U>();
};
```

```
template <typename U>
class Pair<int, U>
{
    public:
        Pair<int, U>(int lhs, U const & rhs) : _lhs(lhs), _rhs(rhs)
        {
            std::cout << "int Partial specialization";
            return;
        }
        ~Pair<int, U>(){}

        int & fst() const {return this->_lhs;}
        U const & snd() const {return this->_rhs;}

    private:
        int _lhs;
        U const & _rhs;

        Pair<int, U>();
}
```

DAY 08: STL

39. STL

CONTAINERS

Sequence Containers

- `std::array`
- `std::vector`
- `std::deque`
- `std::list`
- `std::forward_list`
- `std::basic_string`

Associative Containers

- `std::set`
- `std::multiset`
- `std::map`
- `std::multimap`

Container Adapters

- `std::stack`
- `std::queue`
- `std::priority_queue`

CONTAINERS

Sequence Containers

Sequence containers are container classes that **maintain the ordering of elements in the container**. A defining characteristic of sequence containers is that **you can choose where to insert your element by position**. The most common example of a sequence container is the array: if you insert four elements into an array, the elements will be in the exact order you inserted them.

- `std::array` (C++11)
- `std::vector`: dynamic array capable of growing as needed to contain its elements. The vector class allows random access to its elements via operator[], and inserting and removing elements from the end of the vector is generally fast.
- `std::deque`: is a double-ended queue class, implemented as a dynamic array that can grow from both ends
- `std::list`: a special type of sequence container called a doubly linked list where each element in the container contains pointers that point at the next and previous elements in the list. Lists only provide access to the start and end of the list -- there is no random access provided. If you want to find a value in the middle, you have to start at one end and "walk the list" until you reach the element you want to find. The advantage of lists is that inserting elements into a list is very fast if you already know where you want to insert them. Generally iterators are used to walk through the list.
- `std::forward_list` (C++11)
- `std::basic_string`: can be thought of as a vector with data elements of type char (or wchar).

Associative Containers

Associative containers are containers that automatically sort their inputs when those inputs are inserted into the container. By default, associative containers compare elements using operator<.

- `std::set`: container that stores unique elements, with duplicate elements disallowed. The elements are sorted according to their values.
- `std::multiset`: a set where duplicate elements are allowed.
- `std::map`: (also called an associative array) is a set where each element is a pair, called a key/value pair. The key is used for sorting and indexing the data, and must be unique. The value is the actual data.
- `std::multimap`: (also called a dictionary) is a map that allows duplicate keys. Real-life dictionaries are multimaps: the key is the word, and the value is the meaning of the word. All the keys are sorted in ascending order, and you can look up the value by key. Some words can have multiple meanings, which is why the dictionary is a multimap rather than a map.

Container Adapters

Container adapters are special predefined containers that are adapted to specific uses. The interesting part about container adapters is that you can choose which sequence container you want them to use.

- **std::stack**: is a container where elements operate in a LIFO (Last In, First Out) context, where elements are inserted (pushed) and removed (popped) from the end of the container. Stacks default to using deque as their default sequence container (which seems odd, since vector seems like a more natural fit), but can use vector or list as well.
- **std::queue**: is a container where elements operate in a FIFO (First In, First Out) context, where elements are inserted (pushed) to the back of the container and removed (popped) from the front. Queues default to using deque, but can also use list.
- **std::priority_queue**: is a type of queue where the elements are kept sorted (via operator<). When elements are pushed, the element is sorted in the queue. Removing an element from the front returns the highest priority item in the priority queue.

ITERATORS

An Iterator is an object that can traverse (iterate over) a container class without the user having to know how the container is implemented. With many classes (particularly lists and the associative classes), iterators are the primary way elements of these classes are accessed.

An iterator is best visualized as a pointer to a given element in the container, with a set of overloaded operators to provide a set of well-defined functions:

- Operator* -- Dereferencing the iterator returns the element that the iterator is currently pointing at.
- Operator++ -- Moves the iterator to the next element in the container. Most iterators also provide Operator-- to move to the previous element.
- Operator== and Operator!= -- Basic comparison operators to determine if two iterators point to the same element. To compare the values that two iterators are pointing at, dereference the iterators first, and then use a comparison operator.
- Operator= -- Assign the iterator to a new position (typically the start or end of the container's elements). To assign the value of the element the iterator is pointing at, dereference the iterator first, then use the assign operator.

Each container includes four basic member functions for use with Operator=:

- **begin()** returns an iterator representing the beginning of the elements in the container.
- **end()** returns an iterator representing the element just past the end of the elements.
- **cbegin()** returns a const (read-only) iterator representing the beginning of the elements in the container.
- **cend()** returns a const (read-only) iterator representing the element just past the end of the elements.

It might seem weird that end() doesn't point to the last element in the list, but this is done primarily to make looping easy: iterating over the elements can continue until the iterator reaches end(), and then you know you're done.

Finally, all containers provide (at least) two types of iterators:

- **container::iterator** provides a read/write iterator
- **container::const_iterator** provides a read-only iterator

Iterators provide an easy way to step through the elements of a container class without having to understand how the container class is implemented. When combined with STL's algorithms and the member functions of the container classes, iterators become even more powerful. In the next lesson, you'll see an example of using an iterator to insert elements into a list (which doesn't provide an overloaded operator[] to access its elements directly).

One point worth noting: Iterators must be implemented on a per-class basis, because the iterator does need to know how a class is implemented. Thus iterators are always tied to specific container classes.

```
# include <iostream>
# include <map>
# include <vector>
# include <list>
```

```
class IOperation;
```

```
int main ()
{
```

```
    std::list<int>          lst1;
    std::map<std::string, IOperation*> map1; // version c++ de la hashtable du dictionnaire du tableau associatif
    std::vector<int>        v1;             // comme un tableau qui peut contenir n'importe quoi
    std::vector<int>        v2(42, 100);

    lst1.push_back(1);
    lst1.push_back(17);
    lst1.push_back(42);
```

```
map1["opA"] = new OperationA;
map1["opB"] = new OperationB;
```

```
std::list<int>::const_iterator    it;                // comportement similaire a pointeur
std::list<int>::const_iterator    ite = lst1.end();
```

```
for (it = lst1.begin(); it != ite; ++it)
std::cout << *it << std::endl;
```

```
return 0;
```

```
}
```

algorithmes

algorithmes pré-implémentées pour exécuter des opérations standards

- **Inspectors** -- Used to view (but not modify) data in a container. Examples include searching and counting.
- **Mutators** -- Used to modify data in a container. Examples include sorting and shuffling.
- **Facilitators** -- Used to generate a result based on values of the data members. Examples include objects that multiply values, or objects that determine what order pairs of elements should be sorted in.

Constrained algorithms and algorithms on ranges (C++20)

Constrained algorithms, e.g. ranges::copy, ranges::sort, ...

Execution policies (C++17)

is_execution_policy

execution::seq

execution::par

execution::par_unseq

execution::unseq

execution::sequenced_policy

execution::parallel_policy

execution::parallel_unsequenced_policy

execution::parallel_unsequenced

Non-modifying sequence operations

all_of

any_of

none_of

for_each

for_each_n

count

count_if

mismatch

equal

adjacent_find

search

search_n

lexicographical_compare

lexicographical_compare_three_way

find

find_if

find_if_not

find_end

find_first_of

Modifying sequence operations

copy

copy_if

copy_n

copy_backward

move

move_backward

shift_left

shift_right

transform

fill

fill_n

generate

generate_n

swap

iter_swap

swap_ranges

sample

remove

remove_if

replace

replace_if

reverse

rotate

unique

random_shuffle

remove_copy

remove_copy_if

replace_copy

replace_copy_if

reverse_copy

rotate_copy

unique_copy

shuffle

Partitioning operations

is_partitioned

partition_point

partition

partition_copy

stable_partition

Sorting operations

is_sorted

is_sorted_until

sort

stable_sort

partial_sort

partial_sort_copy

nth_element

Binary search operations

lower_bound

upper_bound

binary_search

equal_range

Set operations (on sorted ranges)

merge

inplace_merge

set_difference

set_intersection

set_symmetric_difference

set_union

includes

Heap operations

is_heap

is_heap_until

make_heap

sort_heap

push_heap

pop_heap

Minimum/maximum operations

max

max_element

min

min_element

minmax

minmax_element

clamp

Permutations

is_permutation

next_permutation

prev_permutation

Numeric operations

iota
inner_product
adjacent_difference
accumulate

Operations on uninitialized storage

uninitialized_copy
uninitialized_move
uninitialized_fill
uninitialized_copy_n
uninitialized_move_n
uninitialized_fill_n

reduce
transform_reduce
transform_inclusive_scan
transform_exclusive_scan

uninitialized_default_construct
uninitialized_value_construct
uninitialized_default_construct_n
uninitialized_value_construct_n
destroy
destroy_n

partial_sum
inclusive_scan
exclusive_scan

destroy_at
construct_at
C library
qsort
bsearch

```
# include <iostream>
# include <algorithm>
# include <list>
```

```
void    displayInt(int i)
{
    std::cout << i << std::endl;
}
```

```
int main ()
{
    std::list<int> lst;

    lst.push_back(10);
    lst.push_back(23);
    lst.push_back(3);
    lst.push_back(17);
    lst.push_back(20);

    for_each(lst.begin(), lst.end(), displayInt);

    return 0;
}
```